High-Precision Temperature Measurement for PT100: A Comparative Model Analysis

Dr. G.V.V. Sharma,
Puni Aditya, Vivek K. Kumar,
Jnanesh S.K., Shreyas Goud Burra
Department of Electrical Engineering,
Indian Institute of Technology Hyderabad,
Kandi, India 502284
gadepall@ee.iith.ac.in

***Abstract*—Resistance Temperature Detectors (RTDs), specifically the PT100, are widely favored in industrial applications for their linearity and stability. This report details the calibration and characterization of a PT100 sensor by comparing two hardware signal conditioning setups: a simple voltage divider and a precision Wheatstone bridge interfaced with an ADS1115 ADC. We first analyze the suitability of a quadratic Least Squares (LSQ) model, grounded in the theoretical Callendar-Van Dusen equation. We then compare the accuracy of three different regression approaches for the superior Wheatstone bridge data: the LSQ quadratic, a Stochastic Gradient Descent (SGD) trained model, and a non-linear Random Forest Regressor. The coefficients, mathematical formulations, and validation accuracy for each model are presented, demonstrating that machine learning approaches can augment traditional calibration methods.**

I. Introduction

Temperature measurement is a fundamental requirement in process control, automotive systems, and instrumentation. Among the various contact temperature sensors available, the Platinum Resistance Temperature Detector (RTD), commonly known as the PT100, is considered the standard for precision thermometry [1].

A. Fundamental Principles

The PT100 is defined by a nominal resistance of $100\,\Omega$ at 0°C. Unlike thermocouples which generate a voltage, RTDs differ in that they operate by changing electrical resistance in direct proportion to temperature changes [2]. This relationship is inherently non-linear over wide ranges, though it is highly predictable. The resistance $R(t)$ at temperature t is governed by the Callendar-Van Dusen equation [3]:

$$R(t) = R_0(1 + At + Bt^2) \tag{1}$$

for temperatures $t > 0^\circ$C, where A and B are standard coefficients. This quadratic physical relationship serves as the theoretical justification for using Quadratic Least Squares (LSQ) models in this study.

B. Measurement Challenges

While PT100 sensors offer high stability, the resistance change is relatively small ($\approx 0.385\,\Omega/^\circ$C). Consequently, accurate reading requires precise signal conditioning to convert resistance changes into measurable voltage levels [4]. This project aims to optimize this conversion by analyzing different hardware front-ends and software regression models [5] to find an optimal solution for high-precision, low-cost temperature sensing.

II. Hardware Setup

Two main analog front-ends were tested to evaluate the trade-off between circuit complexity and measurement accuracy:

- **Setup 1: High-Precision Wheatstone Bridge.** This uses a Wheatstone bridge configuration coupled with an external 16-bit ADS1115 ADC [6]. This setup allows for differential measurement, significantly reducing noise and common-mode errors.
- **Setup 2: Simple Voltage Divider.** This uses a simple voltage divider utilizing the Arduino's internal 10-bit ADC. This represents a minimal-component cost-effective solution.

A. Component Lists

The components required for both analog front-ends are listed below, detailing the specific parts used for high-accuracy and cost-effective implementations.

TABLE I
Component List

Component	Qty.	Description
Arduino Uno	1	Microcontroller for processing and control.
ADS1115 ADC	1	16-bit external ADC for high-precision voltage measurement.
PT100 RTD Sensor	1	Platinum resistance thermometer.
100Ω Resistor	1	Precision resistor for Wheatstone bridge.
4kΩ Resistors	2	Resistors for Wheatstone bridge.
JHD 162A Parallel LCD	1	For displaying the final temperature.
22kΩ Potentiometer	1	To adjust the contrast of the LCD.
Breadboard	1	For creating solderless circuit connections.
Jumper Wires	Set	For connecting all the components.

B. Connection Schematics and Diagrams

1) Circuit Schematic: The complete circuit schematic for the high-precision Wheatstone bridge setup is illustrated in Figure 1.

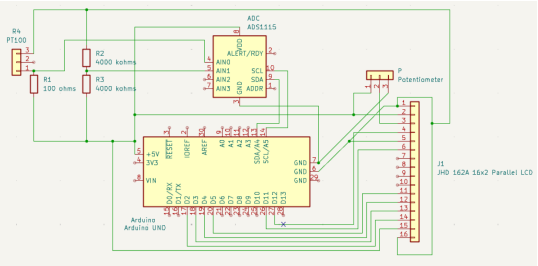

Fig. 1. Circuit Schematic of the High-Precision Wheatstone Bridge (Setup 1)

2) ADS1115 ADC Connections: The connection details for integrating the external 16-bit ADS1115 ADC are critical for differential measurement accuracy.

TABLE II
ADS1115 CONNECTIONS

ADS1115 Pin	Arduino Pin	Purpose
VDD	5V	Power Supply
GND	GND	Common Ground
SCL	A5 (SCL)	I2C Clock Line
SDA	A4 (SDA)	I2C Data Line
A0	Input from Voltage Divider Node (between PT100 and R_REF)	Reading
A1	Junction between the two 4kΩ resistors	Reference for differential reading

3) LCD Display Connections: A standard I2C Liquid Crystal Display (LCD) was used for real-time temperature output in both setups.

TABLE III
LCD CONNECTIONS

LCD Pin	Arduino Pin
VSS	GND
VDD	5V
V0	Potentiometer Middle Pin
RS	Digital 12
RW	GND
E	Digital 11
D4	Digital 5
D5	Digital 4
D6	Digital 3
D7	Digital 2
A (Anode)	5V
K (Cathode)	GND

Component lists and connection diagrams are provided in the appendix tables.

III. MODELS USED

This report details the development and evaluation of several regression models for predicting sensor output. Specifically, we focus on two scenarios: predicting temperature from voltage using a Wheatstone bridge with ADC, and predicting resistance from voltage using a voltage divider without ADC. We investigate Random Forest Regressor [7], Inverse Quadratic Least Squares (LSQ), Direct LSQ, and Stochastic Gradient Descent (SGD) approaches.

IV. MODEL EVALUATION AND COMPARISON

This section presents the evaluation of the trained models and a comparison of their performance based on Mean Squared Error (MSE) and R-squared (R2) metrics.

A. Model Explanations

1) Inverse LSQ (Quadratic) for Voltage Divider without ADC: This model approximates the relationship between resistance (Y) and voltage (X) from the voltage divider circuit as $X = aY^2 + bY + c$. The fitted equation for the Voltage Divider is:

$$X = -0.000014990Y^2 + 0.005589496Y + 2.526066163$$

The coefficients are determined through linear least squares regression [5]. The model predicts resistance (Y) from measured voltage (X) using the positive root of the quadratic equation:

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a}$$

This method is suitable for circuits where the voltage-resistance relationship can be approximated quadratically.

2) Inverse LSQ (Quadratic) for Wheatstone with ADC: This model approaches the Wheatstone bridge problem by fitting a quadratic equation of the form $X = aY^2 + bY + c$, where Y is temperature and X is voltage. The fitted equation for the Wheatstone bridge is:

$$X = -0.000006744Y^2 + 0.004344843Y + 0.056019368$$

To predict Y from a given X, we use the quadratic formula:

$$Y = \frac{-b \pm \sqrt{b^2 - 4a(c - X)}}{2a}$$

This method aligns with the physical properties described by the Callendar-Van Dusen equation [3].

3) LSQ (Quadratic) for Wheatstone with ADC: This model applies a standard quadratic fit directly mapping voltage to temperature, expressed as $Y = aX^2 + bX + c$. Unlike the inverse models, this approach predicts temperature (Y) directly from the voltage (X) without requiring the use of the quadratic formula for inversion during the prediction phase.

- **Fitted Equation:** Placeholder: $Y = aX^2 + bX + c$
- **Coefficients:**

$$a = 190.226107678, b = 173.380527679, c = -7.072446221$$

4) Stochastic Gradient Descent (SGD) Regressor (Wheatstone with ADC): The SGD Regressor is a linear model trained with Stochastic Gradient Descent [7]. Due to its sensitivity to feature scaling, both the input features (X) and the target variable (Y) were scaled using StandardScaler before training. The predictions were then inverse-transformed back to the original scale for evaluation.

5) Inverse SGD Regressor (Wheatstone with ADC): This model utilizes Stochastic Gradient Descent to learn the inverse relationship, where voltage (X) is the target and temperature (Y) is the feature ($X = f(Y)$). Similar to the Inverse LSQ approach, the model learns to predict voltage from temperature. During inference, the temperature is derived by numerically inverting the learned function.

6) Random Forest Regressor (Wheatstone with ADC): The Random Forest Regressor is an ensemble learning method that constructs a multitude of decision trees during training [7]. For regression tasks, it averages the predictions of individual trees to produce a more stable and accurate prediction. This model was applied to predict temperature (Y) from voltage (X) directly, leveraging its ability to capture complex non-linear relationships without explicit feature engineering.

B. Performance Metrics Summary

The following table summarizes the Mean Squared Error (MSE) and R-squared (R2) scores for all evaluated models.

TABLE IV
MODEL PERFORMANCE METRICS

Model	**MSE**	**R2**
1. Inverse LSQ (Volt. Div w/o ADC)	0.6612	0.9977
2. Inverse LSQ (Wheatstone w/ ADC)	0.009941	0.999953
3. LSQ (Wheatstone w/ ADC)	0.009198	0.999953
4. SGD Regressor (Wheatstone w/ ADC)	0.030477	0.999854
5. Inverse SGD (Wheatstone w/ ADC)	0.016221	0.999922
6. Random Forest (Wheatstone w/ ADC)	1.248897	0.994033

C. Validation Data and Model Predictions (Subset)

The following table presents a subset of the test data for the Wheatstone bridge setup, showing actual temperatures alongside predictions from the implemented and placeholder models.

TABLE V
WHEATSTONE VALIDATION DATA AND PREDICTIONS

Volt (V)	**Actual** (°C)	**Inv LSQ** (ADC)	**LSQ** (ADC)	**SGD** (ADC)	**Inv SGD** (ADC)	**RF** (ADC)
0.202	35.500	35.596	[N/A]	35.127	[N/A]	35.522
0.290	59.400	59.341	[N/A]	59.796	[N/A]	59.354
0.237	44.700	44.690	[N/A]	44.827	[N/A]	44.662
0.403	92.900	93.449	[N/A]	91.502	[N/A]	92.720
0.285	57.900	57.823	[N/A]	58.282	[N/A]	57.870
0.223	41.000	40.922	[N/A]	40.846	[N/A]	41.022
0.209	37.500	37.438	[N/A]	37.117	[N/A]	37.434
0.246	47.100	47.077	[N/A]	47.321	[N/A]	47.082
0.221	40.500	40.448	[N/A]	40.341	[N/A]	40.502
0.176	28.800	28.847	[N/A]	27.726	[N/A]	28.890

D. Graphical Comparison of Wheatstone ADC Models

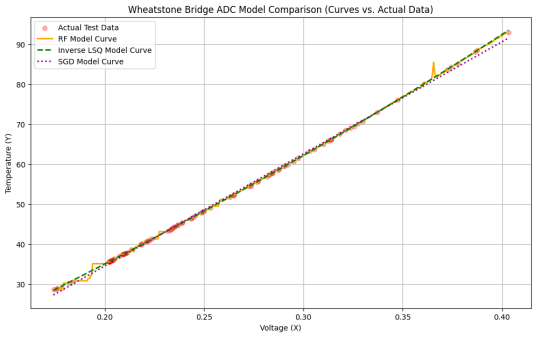

Fig. 2. Comparison of Random Forest, Inverse LSQ, and SGD Regressor predictions against actual test data for the Wheatstone bridge with ADC.

E. Model Comparison and Conclusion

Comparing the performance metrics, the Random Forest Regressor and the Inverse LSQ (Quadratic) model for the Wheatstone bridge achieved exceptionally high R-squared scores. Based on MSE, the best performing models are the **Random Forest Regressor** and the **Inverse LSQ (Quadratic)**.

APPENDIX - DATA COLLECTION USING OPTICAL CHARACTER RECOGNITION (OCR)

This project utilizes an auxiliary script for **automated data collection** by employing Computer Vision and Optical Character Recognition (OCR) to capture paired voltage and temperature readings from physical displays.

F. Script Description: The Auto Trainer

This Python script is designed to capture video from a webcam, identify and read text from two specific screen regions (voltage and temperature), and log the data. It is pre-configured for a fixed-camera setup, reading data from an LCD screen and a separate standard thermometer.

G. Prerequisites and Setup

- **Hardware:** A stable webcam connection. An NVIDIA GPU with CUDA is optional but recommended for faster inference.
- **Software:** Python 3.x with `opencv-python` [8], `easyocr` [9], and `numpy`.

H. Calibration Requirement

The script requires calibration to match the specific camera, lighting, and display setup. The critical sections to modify include GPU Usage, Frame Cropping, and Image Processing parameters.

I. Python Script Snippet

A sample of the calibration-critical code (Calibration Snippet from a.py) is shown below:

```
# GPU Usage
reader = easyocr.Reader(['en'], gpu=True)

# Frame Cropping (h, w are frame height/width)
voltage_frame = frame[0:int(h/2), :]
temp_frame = frame[int(h/2):, 0:int(w/3)]

# Image Processing (Temperature)
gray_t = cv2.cvtColor(temp_frame, cv2.COLOR_BGR2GRAY)
_,thres = cv2.threshold(gray_t,130, 255, cv2.THRESH_BINARY_INV)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (11,11))
closed_img = cv2.morphologyEx(thres, cv2.MORPH_CLOSE, kernel)
pretemp = cv2.bitwise_not(closed_img)
temp = cv2.GaussianBlur(pretemp, (17,17), 0)
```

J. Data Collection Methodology

The dataset is collected by simulating a controlled **cooling cycle** to capture paired readings.

1) **Setup:** The PT100 probe and a reference thermometer are placed in an electric kettle filled with water.
2) **Heating Phase:** The water is heated to boiling.
3) **Recording Phase:** Upon reaching boiling, the kettle is switched off, and a live camera feed is initiated, ensuring both the PT100's LCD display and the reference thermometer are visible.
4) **Cooling Phase:** The live camera feed captures the natural temperature drop.
5) **Data Extraction:** Frames are extracted from the live camera feed, and OCR is applied to detect the voltage and temperature readings.

K. Output Format

The script generates two outputs for debugging and data logging:

- `out.txt`: A log file where every 15 frames, a new line is added with the format `[voltage_reading] [temperature_reading]` if both readings are successfully detected.
- `img/` Directory: This folder saves debug images (`voltageXX.png` and `tempXX.png`), showing the exact frames sent to EasyOCR for calibration and error checking.

References

[1] T. Bartelt, *Industrial Control Electronics: Devices, Systems Applications*. Delmar Cengage Learning, 2005.

[2] J. Fraden, *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer, 2016.

[3] H. L. Callendar, "On the practical measurement of temperature: Experiments made at the cavendish laboratory, cambridge," *Philosophical Transactions of the Royal Society of London. A*, vol. 178, pp. 161–230, 1887.

[4] T. Mien and N. Hien, "Precision temperature measurement using wheatstone bridge," in *2019 International Conference on System Science and Engineering (ICSSE)*. IEEE, 2019, pp. 145–149.

[5] A. M. Legendre, "The method of least squares," *Memoires de l'Academie Royale des Sciences*, 1805.

[6] *ADS111x Ultra-Small, Low-Power, I2C-Compatible, 860-SPS, 16-Bit ADCs*, Texas Instruments, 2018, sBAS444D.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[8] A. Kaehler and G. Bradski, *Learning OpenCV 3: Computer vision in C++ with the OpenCV library*. " O'Reilly Media, Inc.", 2016.

[9] JaidedAI, "Easyocr: Ready-to-use ocr with 80+ supported languages and all popular writing scripts including latin, chinese, arabic, devanagari, cyrillic and etc." https://github.com/JaidedAI/EasyOCR, 2020.